Clean CPU evidence: 0.7.0-clean-aa7347a

Measured source `aa7347a201ce9294b02990b7598af146470bca65`, tree `a2cf4ff5e2bc34249b516316030eb7683d6f3647`; dirty before/after: false. Library source: workspace aa7347a201ce9294b02990b7598af146470bca65. One pinned CPU; f64 operations and explicitly named f32 constructors; row-major inputs.

Workload / library	Per-call p50 / p95 (us)	Batch-mean p50 / p95 (us)	Rust allocations	Peak additional requested bytes	Process peak RSS (MiB)
cholesky_factor_64_f64/faer	8.421 / 8.684	8.375 / 8.742	2	33280	3.92
cholesky_factor_64_f64/kernix	25.074 / 28.426	24.874 / 26.164	2	65536	4.00
cholesky_factor_64_f64/nalgebra	13.479 / 15.926	14.010 / 14.767	1	32768	3.97
cholesky_factor_64_f64/numpy	10.169 / 10.363	10.863 / 11.451	unavailable	unavailable	32.71
create_ones_64x64_f32/candle	0.216 / 0.227	0.227 / 0.267	7	16696	4.65
create_ones_64x64_f32/faer	0.178 / 0.197	0.175 / 0.189	1	16384	3.88
create_ones_64x64_f32/kernix	0.194 / 0.261	0.202 / 0.219	1	16384	3.88
create_ones_64x64_f32/ndarray	0.153 / 0.157	0.148 / 0.159	1	16384	3.96
create_ones_64x64_f32/numpy	1.223 / 1.456	1.323 / 1.434	unavailable	unavailable	32.25
create_zeros_64x64_f32/candle	0.152 / 0.168	0.162 / 0.175	5	16648	4.34
create_zeros_64x64_f32/faer	0.189 / 0.191	0.169 / 0.191	1	16384	4.24
create_zeros_64x64_f32/kernix	0.146 / 0.149	0.139 / 0.152	1	16384	3.95
create_zeros_64x64_f32/ndarray	0.090 / 0.091	0.094 / 0.096	1	16384	3.94
create_zeros_64x64_f32/numpy	0.407 / 0.457	0.396 / 0.416	unavailable	unavailable	32.25
matmul_128_f64/candle	73.667 / 84.133	73.225 / 76.239	9	131352	5.45
matmul_128_f64/faer	80.167 / 98.244	79.309 / 82.204	1	131072	4.67
matmul_128_f64/kernix	115.505 / 140.657	117.954 / 123.555	3	327680	4.66
matmul_128_f64/nalgebra	115.990 / 137.384	118.159 / 129.954	2	327680	4.24
matmul_128_f64/ndarray	113.795 / 137.913	115.164 / 119.112	2	327680	4.32
matmul_128_f64/numpy	76.259 / 89.665	79.038 / 91.551	unavailable	unavailable	33.81
matmul_32_f64/candle	1.505 / 1.544	1.500 / 1.519	9	8592	4.94
matmul_32_f64/faer	1.746 / 2.369	1.499 / 1.674	1	8192	4.32
matmul_32_f64/kernix	1.661 / 1.992	1.649 / 1.776	3	24576	4.35
matmul_32_f64/nalgebra	1.423 / 1.790	1.478 / 1.567	2	24576	3.96
matmul_32_f64/ndarray	1.407 / 1.427	1.475 / 1.628	2	24576	4.01
matmul_32_f64/numpy	2.527 / 2.586	2.553 / 2.695	unavailable	unavailable	32.45
matmul_batched_4x16x32x16_f64/candle	1.991 / 2.116	2.064 / 2.223	25	25504	5.38
matmul_batched_4x16x32x16_f64/kernix	2.061 / 2.929	1.994 / 2.087	7	16392	4.27
matmul_batched_4x16x32x16_f64/numpy	3.004 / 3.189	3.104 / 3.287	unavailable	unavailable	32.51
matmul_sum_backward_32x16x8_32/candle	3.927 / 4.703	4.130 / 4.235	113	27988	5.41
matmul_sum_backward_32x16x8_32/kernix	2.403 / 2.998	2.485 / 2.668	16	18376	4.25
qr_256x64_f64/faer	406.038 / 462.407	429.822 / 492.247	7	331776	4.91
qr_256x64_f64/kernix	496.642 / 563.757	497.196 / 579.007	6	557568	4.84
qr_256x64_f64/nalgebra	363.898 / 406.690	373.939 / 396.122	4	295424	4.29
qr_256x64_f64/numpy	338.857 / 414.165	342.415 / 377.775	unavailable	unavailable	33.61
qr_96x32_f64/faer	70.004 / 76.138	67.234 / 68.743	7	67584	4.55
qr_96x32_f64/kernix	41.632 / 53.167	45.077 / 46.422	6	106752	3.93
qr_96x32_f64/nalgebra	37.427 / 48.281	37.155 / 38.709	4	57600	4.12

Workload / library	Per-call p50 / p95 (us)	Batch-mean p50 / p95 (us)	Rust allocations	Peak additional requested bytes	Process peak RSS (MiB)
qr_96x32_f64/numpy	36.782 / 41.964	39.700 / 42.392	unavailable	unavailable	32.71
softmax_256x64_f64/candle	74.177 / 106.849	76.536 / 80.252	37	265200	5.70
softmax_256x64_f64/kernix	84.400 / 97.076	85.733 / 112.284	1	131072	4.63
softmax_256x64_f64/ndarray	68.581 / 74.825	71.541 / 96.482	1	131072	4.41
softmax_256x64_f64/numpy	105.246 / 122.622	108.770 / 112.975	unavailable	unavailable	32.71
training_backward_2x4x8x4_f64/candle	13.301 / 16.653	14.207 / 15.088	472	8332	5.93
training_backward_2x4x8x4_f64/kernix	8.668 / 9.696	9.040 / 9.681	71	4224	4.41
training_forward_2x4x8x4_f64/candle	3.282 / 3.471	3.422 / 3.614	126	3912	5.68
training_forward_2x4x8x4_f64/kernix	3.809 / 4.958	4.210 / 4.488	34	1040	4.16
training_record_2x4x8x4_f64/candle	4.756 / 5.521	5.150 / 5.475	149	10712	5.77
training_record_2x4x8x4_f64/kernix	4.646 / 5.603	4.753 / 4.847	39	6832	4.35
training_sgd_2x4x8x4_f64/kernix	0.483 / 0.510	0.515 / 0.540	5	688	4.57

Method and limits

- Per-call quantiles use nearest ranks over 256 individual warmed calls; batch quantiles use 15 means over separate 10-ms batches. They are separate distributions. Raw samples remain in raw.json.
- Eight warmup calls follow validation. Timing includes allocation, dispatch, computation and output destruction. The first call field excludes output destruction. Inputs and library/runtime imports are prepared outside timing.
- Three extra processes record their first operation calls. These are first-use observations, not cache-flushed or device-startup latency. Timer-pair overhead is reported without subtraction; tiny calls are clock-sensitive.
- ndarray and Candle softmax are explicit stable compositions, not claims about a native softmax API.
- Constructor cases allocate owned 64x64 f32 tensors. Pre-0.6 registry baselines use full(value), explicitly marked in metadata; current workspace runs use zeros/ones. No nested literal reinterpretation is involved.
- All GEMM inputs are C-order and match elementwise. Native output layout differs by library. QR materializes thin Q and R in every implementation. Cholesky returns reusable factors (or NumPy L); no extra factor materialization is timed.
- Kernix checks finite values/symmetry. Some competitors validate a narrower contract. No finite-data precondition check was removed to improve timing. Accuracy results and budgets are retained per worker.
- The backward case uses an actual reverse-mode graph in Kernix and Candle for sum(A @ W), with A constant and W trainable. Both copy input/weight data and returned gradients; no manual derivative is used as a framework ranking.
- Allocations are measured in a separately compiled instrumented binary after warmup, outside all timing loops. Counts cover Rust global-allocator requests, not native allocator internals, stack memory or all NumPy allocations. NumPy allocation counts are unavailable, not zero.
- Requested-byte peak is additional live Rust allocation during one call, including output; it is not scratch-only memory or peak RSS. RSS is the process high-water mark including fixtures, validation and runtime, read from Linux VmHWM. Other platforms may report unavailable.
- The published cases are allocation-inclusive CPU APIs and graph workloads. Pure internal kernel-only and end-to-end application/model latency are not measured. No workload result implies a complete application frame rate.
- faer is explicitly serial; Rayon and BLAS thread counts are one. No GPU comparison. Python/NumPy timings include Python dispatch. Backend and dependency versions/configuration, governor and RAM are recorded.
- Training stages use batched matmul + bias + layer_norm + tanh + step-two feature slice + MSE (Candle index_select). Forward excludes graph recording; record includes parameter/input/target snapshots and eager forward/loss; backward reuses a prepared graph and returns fresh derivatives. Allocation probes measure the corresponding stage, including graph values/scratch, not graph-container-only bytes. SGD uses fixed precomputed real gradients and includes transactional validation/parameter updates; it is not an end-to-end converging training loop. Candle normalization is an explicit population-statistics composition; its

checks/rounding differ. Historical registry baselines without these Tape operations omit the stages, recorded in metadata, rather than substitute manual derivatives.

- A shared laptop has uncontrolled load/frequency/cache effects. Report comparator wins and losses; no universal ranking follows. The source archive comes from committed bytes, with result directories excluded. Library dependencies are pinned by the saved driver lockfile.